Scenario Resolver

A guided reading of `resolve_scenario.py`

Master's Thesis — *Asset-Centric Temporal Graphs for Crude-Oil Logistics: Schema Design and Consistency Guarantees*

Pedro Porfirio · NOVA IMS · Universidade Nova de Lisboa

Supervisor: Professor Flavio Pinheiro

Document: **Resolver Walkthrough** · v1.0 · 2026-05-15

1. Why the resolver exists

Before the resolver, every consumer that wanted to read a variable's value had to re-implement resolution logic itself. The balance UI carried a 100-line CTE handling TS bindings, mirror formulas, reverse-paired-edge inheritance, and closure-equation evaluation. JavaScript walked arithmetic formulas at render time, because SQL could not evaluate multi-term expressions of the form `A − B − C`. Each new generator (hierarchy explorer, geographic map, Chapter-5 validation queries, future forecasting features) would have duplicated that work.

The resolver collapses every consumer's resolution logic into a single deterministic pass:

```
inputs   :  variables  +  variable_assignments  +  timeseries_data
            +  the active scenario

output   :  one row per (variable, date) in scenario_resolved_values
```

Every downstream consumer becomes a thin `SELECT` over the result table. The resolver runs in roughly twenty seconds for the starter scenario (1,830 variables × 156 monthly observations); it is run once per assignment change, not once per consumer.

2. The output table

The resolver writes to `oil_network.scenario_resolved_values`. Two columns from the DDL deserve particular attention:

```
CREATE TABLE oil_network.scenario_resolved_values (
    scenario_id       TEXT NOT NULL  REFERENCES scenarios     ON DELETE CASCADE,
    variable_id       TEXT NOT NULL  REFERENCES variables     ON DELETE CASCADE,
    observation_date  DATE NOT NULL,
    value             DOUBLE PRECISION,
    source            TEXT NOT NULL CHECK (source IN
                        ('observed', 'derived', 'zero', 'latent', 'unresolved')),
    formula_used      TEXT,
    timeseries_id     TEXT,
    saved_date        TIMESTAMPTZ DEFAULT NOW(),
    run_id            BIGINT REFERENCES scenario_resolver_runs,
    PRIMARY KEY (scenario_id, variable_id, observation_date)
);
```

The `value` column is nullable. A NULL value paired with `source = 'latent'` is a valid and intentional row — it tells downstream consumers "this variable is declared but its value is unknown by design; do not retry." This is the schema-level expression of Principle 2.11 (latent allocation at junctions): the resolver records that a flow exists structurally and is constrained by mass balance, but its per-edge value is not pinned down by observation.

The `source` column is a CHECK-constrained enum with five categories:

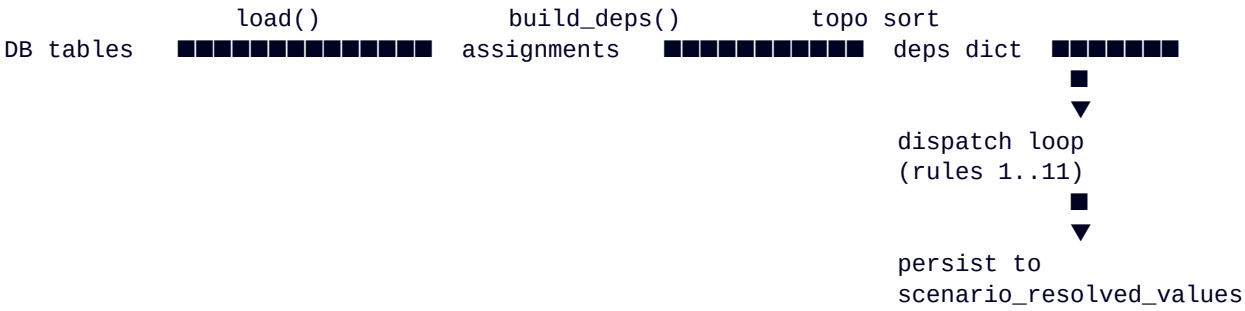
source	meaning
observed	Direct time-series lookup — ground-truth measurement.
derived	Value computed by a formula (alias, sum, closure, arithmetic, mirror).
zero	Structural zero (e.g. a refinery does not produce crude).
latent	Declared variable with no resolution path (value = NULL by design).
unresolved	A formula was given but could not be evaluated. Zero in a healthy run.

`formula_used` preserves which resolution rule fired (the dispatch labels in Section 4). This is for auditing: it lets a reviewer reconstruct, for any value, the exact path that produced it.

Every run also writes an audit row to `scenario_resolver_runs` with a started/completed timestamp, duration, dispatch counts as JSONB, and optional free-text notes. Every resolved-value row carries the `run_id` that produced it, so historical comparison is a single SQL join.

3. How a run flows, end to end

The top-level shape of `resolve()` follows five steps:



Each phase has one purpose, isolated from the others. The dispatch loop never reads from the DB; the loader never evaluates formulas; the topological sort never produces values. Mistakes in one phase are diagnosed by inspecting its single concern.

3.5 Where the resolver starts

A common framing of the resolver is that it starts from "production nodes with *P* set and outflow equal to production." That is a special case, not the rule. The actual starting points are the **leaves of the dependency DAG** — every variable with no upstream dependency. There are two categories:

(a) Observed variables. Any variable with `timeseries_id IS NOT NULL`. The TS lookup is the ground truth; no other rule contributes to its value. Examples: `production__crude__bakken_nd` (EIA series `COPRPM`), `consumption__crude__padd3_view` (refinery inputs).

(b) Structural zeros. Any variable with `formula = '0'`. These come from `node_type_default_formulas` — a pipeline's *P*, *C*, Δ*S*, and *B* are zero by physical construction; a refinery's outflow is zero (refineries consume crude, they do not return it). No upstream variable contributes to a structural zero either.

Everything else is derived from these two via the topologically ordered dispatch loop. The propagation works bottom-up: an alias needs its target resolved first; a sum needs its inputs; a closure needs the inventory delta and the flows. The topological sort guarantees each variable is visited only after its dependencies are.

For a basin production node like `bakken_nd`, the chain is: *P* observed → *C*, Δ*S*, *B* = 0 (structural defaults) → *O* alias-bound to *P* (single-outflow fanout). That is the original framing's "O = P", but the rule that fires is rule 5 (alias), and the special-case framing breaks for multi-outflow nodes (rule 5 cannot fire; *O* stays latent).

3.6 A common misconception: "o1 = i2 + i3"

The framework does **not** automatically aggregate the outflow of one node from the inflows of its downstream neighbours, even when the routing topology would seem to imply it. Each directed flow is its own variable. The pair `outflow__crude__permian_tx__permian_tx_gathering` and `inflow__crude__permian_tx_gathering__permian_tx` are *two different variables* that happen to refer to the same physical edge. They are linked by the **reverse-mirror rule** (Rule 6) — if one is observed or derived, the other inherits from it — not by summation.

The "*o*1 = *i*2 + *i*3" pattern is different: it is a *node-level mass balance constraint*, not a sum-of-variables formula. At a gathering node, the resolver does not derive any one outflow from the sum of inflows. What does the framework offer instead?

- **v_node_balance_check** (L4a, refreshed per resolver run) exposes `sum_in`, `sum_out_obs`, `sum_out_implied`, and `gap_kbd` for every (node, date). The mass-balance constraint is a queryable artefact, not a propagating formula.
- When the per-edge split is not observable in public data (e.g. the three Permian-to-Gulf outflow routes), each edge stays *latent*. The aggregate constraint Σ*O* = Σ*I* still holds in v_node_balance_check; the LP exporter (future work) reads the latents as decision variables and the constraint as an equality row.
- Aggregation in the other direction — *parent* = Σ *constituents* — *is* a propagating formula: declared at construction time via `formula_inputs` on the parent's assignment, then applied by rule 4 (sum) at resolution time. Aggregation parent-to-children, not edge-to-edge.

3.7 Latent vs unresolved

Both leave `value = NULL`, but they mean different things and the framework treats them differently. The distinction matters for downstream consumers and especially for the future LP / optimisation layer.

source	Meaning	How a downstream consumer should treat it
`latent`	Declared variable, no resolution path. The assignment is `formula = 'latent()'` and no reverse-mirror promotion fired. Value is unknown by design.	Free variable in an optimisation; constrained by mass balance, capacity, and observations that reference it. Typical use: per-route flow at a fan-out junction (Principle 11).
`unresolved`	A formula was given but the resolver could not evaluate it. Either a referenced variable is missing or the formula pattern is unmodelled. *Zero rows expected in a healthy run.*	A bug. Fix the assignment (correct the reference) or extend the resolver (add a rule). Never silently treated as latent.

4. The dispatch loop: ten resolution rules

Rules are tried in priority order. The first rule that succeeds produces a value and short-circuits the rest. Rule 11 (unresolved) is a fallback that should fire zero times in a healthy run.

The full set of canonical rule labels — that is, the only `source` values that `scenario_resolved_values` ever records — is short. Three earlier labels were collapsed into the canonical `sum` in the twelfth pass; they no longer appear in the codebase or the resolved table:

Canonical label	Retired predecessors
`observed`	—
`zero`	—
`latent`	—
`sum`	`sum_over_children`, `sum_over_outflows`, `sum_same_type` — semantic role recoverable from `formula_inputs` structure, see Design_Principles § The canonical sum rule
`alias`	—
`reverse_mirror`	—
`arithmetic`	—
`closure`	dormant in starter scenario after B promoted to TS-observed (sixth pass)
`unresolved`	fallback; zero rows expected

The full per-rule details follow.

#	Rule	Description
1	**Observed**	`timeseries_id IS NOT NULL`. Direct TS lookup. This is the ground truth; every other rule is downstream of these. In the starter scenario, 80 variables fire this rule.
2	**Structural zero**	`formula = '0'`. Used for refinery production, pass-through inventory, and other slots where the value is zero by physical construction. Roughly 628 variables fire this rule, mostly non-relational scalars inherited from `node_type_default_formulas`.
3	**Latent, with reverse-mirror promotion**	`formula = 'latent()'`. Records value = NULL and source = 'latent'. But first, the rule attempts a *reverse-mirror promotion*: if the paired direction (opposite variable_type, swapped endpoints) has a resolved value, the rule borrows it via the mirror identity. Without this promotion, a latent `outflow_to_padd2` on `canadian_oil_sands` would beat the reverse-mirror rule and leave the variable NULL despite a known value on the inflow side of the same edge.

#	Rule	Description
4	**Sum**	`formula = 'sum'`. Sums every value in `formula_inputs`; NULL inputs are skipped. This is the twelfth-pass collapse of three earlier labels (`sum_over_children`, `sum_over_outflows`, `sum_same_type`) into one canonical rule. The semantic role — aggregation parent, fan-out total, residual — is recoverable from the structure of `formula_inputs` (same-type and same-commodity ⇒ aggregation; mixed-type at same node ⇒ fan-out), so the formula text does not need to encode it.
5	**Single-variable alias**	`formula = bare_variable_id` and the `formula_inputs` list contains at most that one entry. The variable inherits its value from the named target. Most aggregate-to-aggregate alias chains fire this rule; in the starter scenario, 442 variables resolve via alias.
6	**Reverse mirror**	Relational variable with no own resolution; its paired direction (opposite type, swapped endpoints) has a resolved value. The variable borrows that value. After the ninth-pass fix, this rule fires deterministically — `build_deps()` now guarantees the paired side is resolved before this side is evaluated, so 15 mirrors fire reliably (previously 3 by luck of insertion order).
7	**Closure formula**	Triggers when the variable is a balancing_item whose formula_inputs include a mix of inventory, inflow, and outflow variables. Evaluates `B = ΔS − P + C − ΣI + ΣO`. Now dormant in the starter scenario because B was promoted to TS-observed in the sixth pass (MCRUA series).
8	**Arithmetic combination**	Parses the formula text as signed variable references (`A − B + C`). Used for residual definitions: `padd2_other.P = padd2_view.P − bakken_nd.P − oklahoma.P`.
9	**Unresolved**	Records value = NULL, source = 'unresolved'. The fallback. In a healthy run this fires zero times; non-zero counts indicate a bug or an unmodelled formula pattern.

5. Building the dependency DAG

`build_deps()` walks every assignment and produces a dict `deps[variable] = set of upstream variable_ids`. The set must be acyclic so that `graphlib.TopologicalSorter` can produce a valid evaluation order. The build has three sources of dependencies:

(a) formula_inputs. Every variable id listed in `formula_inputs` that exists in the assignment set becomes a dependency.

(b) sum_over_outflows. The variable depends on every outflow variable at the same node, except itself.

(c) Reverse-mirror dep, hoisted above the early continue. When a variable is latent and its paired direction is resolved, the paired direction is added as a dependency. Two cycle-avoidance gates protect this: (i) the paired side must not itself be plain latent, else neither side has a value; and (ii) the paired side must not already alias from us, else we get a us ↔ paired loop. The second gate is the `paired_aliases_us` check added in the ninth pass.

Cycle gates matter because the topological sort throws `CycleError` on the first cycle. The resolver tolerates many alias and mirror patterns precisely because of these gates; the cost is a handful of subtle conditions in `build_deps()` that need to be read together.

6. The closure formula

Rule 8 evaluates the mass-balance closure when the parent variable is a balancing_item whose inputs span the requisite variable types:

```
B = ΔS  −  P  +  C  −  ΣI  +  ΣO

ΔS_kbd  =  (S(t) − S(t−1)) / days_in_month(t)     # kbd, not MBBL

If any required input is NULL at date t, the closure is skipped
for that date — the resolver does not silently substitute zero.
```

Skipping rather than zero-imputing matters: a NULL inflow with a non-zero P and C would produce a spurious B if treated as zero, and that B would propagate into the USA aggregate. Skipping preserves the signal that some upstream value is genuinely missing.

Since the sixth pass, B at PADD and USA level is TS-observed (MCRUA series), so closure is no longer the primary path. It remains as a fallback constraint — particularly valuable for the Chapter-5 Claim-4 view, which compares closure-derived B against observed B and reads the delta as the magnitude of partition-internal latent flow.

7. Arithmetic formula evaluator

Rule 9 parses formula strings of the form `A − B − C` via a small regex that pulls out signed terms. Each term must match a variable id present in `formula_inputs`; anything that fails to match aborts the rule cleanly and falls through to the next rule. This limits the surface for parser exploits and keeps the formula language human-readable.

```
RE_TERM  =  re.compile(r'([+-]?)\s*([a-z][a-z0-9_]*)')

for m in RE_TERM.finditer(formula):
    sign = -1 if m.group(1) == '-' else 1
    tok  = m.group(2)
    if tok not in input_set:
        ok = False; break
    terms.append((sign, tok))
```

Eight variables in the starter scenario fire this rule. They are structural residual definitions such as `padd2_other.P = padd2_view.P − bakken_nd.P − oklahoma.P`, where the residual exists precisely to soak up the difference between an aggregate total and its enumerated constituents.

8. Persistence and the audit trail

After the dispatch loop finishes, the resolver clears the scenario's existing rows in `scenario_resolved_values` and writes the new ones via `execute_values` in pages of 5,000. The audit row in `scenario_resolver_runs` is updated with the completion timestamp, the elapsed milliseconds, and the dispatch counts as JSONB.

If the resolver crashes mid-way, the open audit row with NULL completed_at is itself a useful signal — it tells the next operator that a run was started but did not finish. The resolved-values table is unaffected by a crash because the DELETE/INSERT pair runs inside a transaction.

9. Bugs encountered and fixed

9.1 The latent-mirror cycle

Original implementation: a latent variable would add its paired direction as a dependency unconditionally. When the paired direction was also latent and aliased to its own paired side, this produced a two-node cycle (us → paired → us) and TopologicalSorter raised `CycleError`. Fix: require the paired side to be non-latent before adding the dep (first cycle gate).

9.2 The latent short-circuit that hid reverse-mirror

Rule 3 (latent) ran before Rule 7 (reverse-mirror), so a variable explicitly marked latent would record value = NULL even when its paired direction had an observed value. Fix: promote the mirror inside Rule 3 — if the paired side is resolved, borrow its value first and only fall through to NULL otherwise. Together with the build_deps hoist (so that latent variables get their paired-side dep added before being skipped), this lifted mirror dispatch from 3 (by luck of insertion order) to 15 (deterministic).

9.3 The alias-rule wrapper bug

Rule 6 matches when `formula in by_id`, i.e. the formula text is itself a bare variable id. Earlier migration scripts wrote `formula = 'alias(variable_id)'` with the wrapper string; rule 6 missed those, and they slipped past on Rule 10 (same-type rollup) by coincidence. The fix is in the migration scripts: `formula = bare_variable_id`, with the wrapper retained only as the display label in `formula_used` on resolved rows.

10. Dispatch counts in the current starter scenario

Live state at PDF generation: run_id = 1, completed 2026-05-15. 223,114 rows persisted to `scenario_resolved_values`.

Rule	Count
observed (TS lookup)	80
zero (structural)	628
latent (incl. reverse-mirror failures)	652
alias (single-variable)	442
reverse-mirror (promotion + Rule 6)	15
arithmetic	8
sum (canonical, collapses sum_over_*/sum_same_type)	5
closure (B = ΔS – P + C – ΣI + ΣO)	0
unresolved	**0**

Zero unresolved is the load-bearing invariant. Every declared variable has a value or an explicit latent marker. The downstream consumers can rely on the presence of a row for every (variable, date) pair, and on the source column telling them whether the NULL is by design (latent) or by failure (unresolved, which is an unmodelled case that needs attention).